# ISSUES IN DESIGN OF A CODE GENERATOR

* While design the code generator, there are several problems encountered.
* The issues are as follows.

i) Input to the code generator
ii) Target program
iii) Memory Management
iv) Instruction Selection
v) Register Allocation
vi) Choice of Evaluation Order.

## Input to the code Generator:

* The input sent to the code generation phase is also important while designing the code generator.
* The input consist of the intermediate representation of the source program produced by the front-end along with symbol table for finding the addresses of the data objects present in the intermediate representation.
* Even though many intermediate representation ~~of the source program produced~~ are there (postfix, quadruples, syntax trees, DAGs).
* It is the basic assumption that its input is free from errors.
* The values of names appeared in the intermediate language are represented in terms of quantities that the machine can directly manipulate.
* And necessary type conversion is also performed with the inputs.

## Target Programs.

* The output of the code generator may be in a variety of form as,
  i) Absolute machine language
  ii) Relocatable machine language
  iii) Assembly language

i) For the first case, the target program is placed in a fixed memory location and executed immediately.
→ Smaller programs compiled and executed quickly.

Eg: C Compiler produces absolute code as the target program.

ii) For the relocatable machine language, the compiler allows sub programs to be compiled separately.
* It needs loader and linker for explicit loading and linking.
* Even though there is an expense on linking and loading, there is a large flexibility in compiling subroutines separately and to call other previously compiled program.
* If relocation is not there in the compiler, explicit relocation information has to be sent to the loader to link separately compiled program.

iii) For the assembly language program, the process of the code generation is easier.
* It is easy to use symbolic instructions and macro facilities of the assembler.
* It works well for machines with smaller memory.

## Memory Management:

* It addresses the problem of mapping names in the source program to address of data objects in run-time memory by the front end as well as the code generator.

* The symbol table entries corresponding to names are created whenever there is a declaration in the procedure.

* By seeing the type field of each variable, the amount of storage required is identified accordingly the relative address can be determined.

* While generating the machine code, the labels in the 3 address statements have to be converted into required address of instructions. (Called as backpatching).

* While scanning each quadruple, a count has been maintained for storing the list of symbol used.

Instruction Selection:

* The instruction set of the target machine decides the instruction selection.

* The target machine supports all data types only if the instruction set follows uniformity and completeness.

* Likewise instruction speeds and machine idioms are also matters while designing the compiler.

* For each type of three-address statement, a skeleton code can be generated.

For example,

* A three-address statement of the form

$$x := y + z$$

where $x, y, z$ are statically allocated is translated into a code sequence.

```
MOV  y, R0    // R0 ← y
ADD  z, R0    // R0 ← R0 + z
MOV  R0, x    // x ← R0
```

* But statement-by-statement code generation produces redundant codes.

For eg.,

```
a   b+c
d   a+e
```

* The translated code sequence is as follows

```
MOV  b, R0    // R0 ← b
ADD  c, R0    // R0 ← R0 + c
MOV  R0, a    } unnecessary moves
MOV  a, R0    }
ADD  e, R0    // R0 ← R0 + e
MOV  R0, d    // d ← R0
```

* The fourth statement is redundant because it is moved to the memory variable only in the last step.
* Thus the quality of the generated code is based on the speed and size of the code.
* Likewise the way of implementing an operation with suitable instruction, decides the efficiency of a code generator

For example,

↳ the three instructions for incrementing a variable can be replaced by a single INC instruction.

```
MOV  a, R0   }
ADD  #1, R0  } is equivalent to INC a
MOV  R0, a   }
```

* Depending upon the context, the machine code sequence is identified.

## Register Allocation:

* The registers in the target machine affects the design of code generation.
* The instructions involved with register operands are faster that those involved with memory operands.
* Proper usage of registers is important in generating codes.
* The use of register involves 2 phases called as register allocation and register assignment.
* In register allocation phase, the set of variables residing in registers at a point has to be selected.
* In register assignment phase, the specific register that a variable resides in has been picked.
* It is difficult to assign registers for variables.
* Because the hardware and operating systems may require certain register-usage conventions.
* Some machines uses registers-pairs for storing the operands and result.

For eg., the IBM Systems/370 machine uses registerpairs for integer multination.

M X, Y

Where, M – Multiply instruction

X – Multiplicand value taken from even register

Y – Multiplier value taken from odd register.

* Consider the two - three address statement below:

| | |
|---|---|
| $t_0 := a + b$ | $t_0 := a + b$ |
| $t_0 := t_0 * c$ | $t_0 := t_0 + c$ |
| $t_0 := t_0 / d$ | $t_0 := t_0 / d$ |

Fig: Three address Codes.

* The optimal code sequences for the three address code generated are as follows.

```
L   R1,a   // Load a to R1
A   R1,b   // R1 ← R1+b
M   R0,c   // R0 ← R0*c
D   R0,d   // R0 ← R0/d
ST  R1,t
```

* The resultant code is given by,

```
L     R0,a
A     R0,b
A     R0,c
SRDA  R0,32   // Shifts the dividend into
                 R1 and clears R0.
D     R0,d
ST    R1,t
```

## Choice Of Evaluation Order:

* The order of evaluating the expressions is also a factor affecting the efficiency of the target code.
* Identifying the best order is an optimization problem.

* Many problems of this kind are NP-complete problems.

## A SIMPLE CODE GENERATOR ALGORITHM:-

* A code generator generates target code for the three-address code segments.
* By considering the operands currently in registers, the target code is generated and keeping the results in registers except it is required by another computation or just before a procedure call or jump statements.
* While generating the target code, several cases to be considered for the expression

$$X := Y + Z$$

i) Whether Y, Z are stored in register? If it is so, generate the code,

```
ADD Rj, Ri
```

ii) Only one variable in register. The target code can be

```
ADD Z, Ri
(OR)
MOV Z, Rj
ADD Rj, Ri
```

* This algorithm needs descriptions to keep track of register contents and addresses for names.

i) Register Descriptor

* It keeps track of the value currently in each register.

* When a register is required, the compiler consults register descriptor whether it is empty or holding any value.
* The register descriptor contains the list of variables whose current information is available in that register.

ii) Address Descriptor:

* It keeps of the locations where the current value of the name is found at run-time.
* This lists the locations of register, memory or stack where the variable is found.

Code generation algorithm:

* The code generated for the three address statement x := y op z involves the following steps-

1. Invoke getreg() function which returns L i.e, the location where the result of the operation is to be stored.
* L may be a register or a memory location.
2. Check the address descriptor content of y; the current location of y and find out where the value of y is available.
* Select register location of y if the value of y is currently both in memory and a register.
* Then generate the instruction MOV y', L if L does not have y already.

3. Consult z', the address descriptor of z to find out where the current value of z is available.
* Prefer the register location z' which returns both. Generate the instruction op z', L.

4. Update the address descriptor of x to have the value of L.
* If L is a register, then update the register descriptor of L to keep x.
* If L is in the address descriptor of y, remove y from it.

5. If z has no next use in the block and z' has a register location then free the register occupied by z.
* If the address descriptor of z has only register location and z has no next use, generate the instruction MOV R, z to update M of z to hold the current value.
* If z' has both register and memory location, no need to generate a MOV instruction since the current value is already in memory location.

→ The code generate for the unary operation eg. x := y is as follows.
i) check the address descriptor of y, if it returns ~~retu~~ a register and memory location, then update the register descriptor to have x also, change the address descriptor of x

to show only the register location.

ii) If the register descriptor of y returns only a memory location, then generate MOV y', R.

* Update the address descriptor of y also to have R.
* Now the register descriptor is modified to have x, y and address descriptor of y to have R.

iii) At the end, the current value of all live variables that live on exit are updated in their memory locations by appropriate MOV instructions.

For eg.,

a := b+c

* The code generated for this three address code is as follows:

```
MOV  C, Rj   // cost=2
ADD  Rj, Ri  // cost=1.
```

## The getreg() function:

* The getreg() functions return the location L to hold the value of x for the assignment statement x := y op z.
* The choice of L either it is a register or a memory location is identified by the following algorithm.

1. If y is in a register and y is not live and has no next use, then return register of y for L.

* And also update the address descriptor of y to inform that y is not there in L.

2. If y is not in a register, return an empty register for L if it is there.

3. Else forcibly free a register and return it as L.

* choose a register which has minimum identifiers in its descriptor or chose a descriptor whose values shows furthest information in liveness.

4. Else return the memory location of x as L.

Eg: Convert the following three-address code into assembly code.

Statement: d := (a-b) + (a-c) + (a-c)

3 address code:

$t_1 := a - b$

$t_2 := a - c$

$t_3 := t_1 + t_2$

$d := t_3 + t_2$

Based on the code-generation algorithm

i) $t_1 := a - b$

generated code:

MOV a, $R_1$ // $R_1 \leftarrow a$

SUB b, $R_1$ // $R_1 \leftarrow R_1 - b$

Register Descriptor:

| | |
|---|---|
| $t_1$ | |

$R_1$

Address Descriptor:

| |
|---|
| $t_1 \rightarrow R_1$ |

ii) $t_2 := a - c$

MOV a, R2 // $R2 \leftarrow a$

SUB c, R2 // $R2 \leftarrow R2 - c$

Register Descriptor:

| | |
|---|---|
| t1 | R1 |
| t2 | R2 |

Address Descriptor:

| | |
|---|---|
| $t_1 \rightarrow R_1$ | $t_1 \rightarrow R_2$ |

iii) $t_3 := t_1 + t_2$

ADD R2, R1 // $R1 \leftarrow R1 + R2$.

Register Descriptor:

| | |
|---|---|
| t3 | R1 |
| t2 | R2. |

Address Descriptor:

| | |
|---|---|
| $t_3 \rightarrow R_1$ | $t_2 \rightarrow R_2$ |

iv) $d := t_3 + t_2$

ADD R2, R1 // $R1 \leftarrow R1 + R2$

MOV R1, d // $d \leftarrow R1$

Register Descriptor:

| | |
|---|---|
| d | R. |

Address Descriptor:

| | |
|---|---|
| $d \rightarrow R_1$ | $d \rightarrow L$ |

Some memory location.

# Code Generation for other Statements

| | Statement | Code. |
|---|---|---|
| i) | a := b[i] | MOV b(Ri), R |
| ii) | a[i] := b | MOV b, a(Ri) |
| iii) | a := *p | MOV *Rp, a |
| iv) | *p := a | MOV a, *Rp |
| v) | If x<y goto z | CMP x, y<br>CJ< z |